

Privacy-Preserving Face Recognition via Federated Learning

A distributed face recognition framework that integrates MTCNN-based preprocessing, InceptionResNetV1 feature extraction, and DP-SGD training under a federated architecture. The system is evaluated against model inversion attacks, with biometric data remaining on-device throughout the training process.

Authors

Abderahim Khefif Kossai Baha Afaf Khadraoui
Amel Saidouni Mehdi Bouzoul

Abstract

Biometric face recognition is convenient, but the usual centralized pipeline forces people to hand over photos to a server that becomes a single point of privacy failure. This project addresses that problem with a federated face recognition system in which raw images stay on the device while only model updates move between clients and server. The implementation combines MTCNN preprocessing, InceptionResNetV1 embeddings, Flower-based federated averaging, Opacus differential privacy, Ed25519 signatures, AES-256-GCM encryption probes, and tamper-evident audit logging. We evaluate the system through model inversion experiments, using two model versions: a vulnerable baseline without differential privacy and a protected version trained with DP-SGD. The main empirical result is a consistent two-orders-of-magnitude inversion-loss separation between the two versions across two target identities, showing that the DP model is substantially harder to reconstruct. The report is also honest about the implementation limits: the backbone was frozen to fit the workload on student laptops, which kept the local embeddings stable but prevented meaningful federated improvement, and the privacy accountant exported a cumulative epsilon of 10.21 even though the configured cap was 5.0. These limitations do not invalidate the privacy demonstration, but they matter for how the system should be interpreted.

I. INTRODUCTION

Face recognition is already embedded in daily life. It unlocks smartphones, supports border control, and simplifies access control in workplaces and campuses. The convenience comes with a serious cost: most commercial systems work by sending face photos to a central server, and once those images are collected the privacy risk is permanent. Unlike a password, a face cannot be changed after a breach.

Federated learning offers a more careful alternative. Instead of moving photos to the server, the model is trained across devices and only the weight updates are shared. McMahan et al. showed that this idea can train deep models efficiently with FedAvg, and later surveys have refined the practical challenges around non-IID data, communication cost, and system heterogeneity [1], [6], [7]. In this project, federated learning is the architectural starting point, not the endpoint.

The complication is that federated learning alone is not enough. A model can still leak information through its weights, even when raw photos never leave the device. Inversion attacks make that risk concrete: an attacker who obtains the trained model can try to recover what kind of face would activate the model for a specific identity. Fredrikson et al. showed that such attacks are practical when model outputs or parameters leak too much information [3]. The important lesson is that privacy is not guaranteed by data locality alone.

This project therefore implements a layered face recognition system. It keeps biometric data local, trains with federated learning, adds differential privacy to reduce reconstruction leakage, and complements that with digital signatures, encryption primitives,

and audit logging. The system is designed as a complete story: preprocessing, local training, privacy defense, attack evaluation, and frontend integration. The rest of this paper is organised as follows. Section II reviews the relevant background, Section III describes the architecture, Section IV details the implementation pipeline, Section V explains the privacy and security stack, Section VI evaluates the attacks, Section VII summarises the frontend, Section IX discusses limitations, Section X outlines future work, and Section XI concludes.

II. BACKGROUND AND RELATED WORK

A. Federated Learning

Federated learning trains one model across multiple devices without centralising the underlying data. In FedAvg, each client starts from the current global weights, performs local updates on its own data, and sends the resulting parameters back to the server for aggregation [1]. This reduces privacy exposure, but it also introduces new problems: client data are often non-IID, devices differ in memory and compute, and communication constraints can dominate runtime.

B. Differential Privacy

Differential privacy formalises how much information about a single record can be inferred from a training procedure [2]. In deep learning, the standard implementation is DP-SGD: each gradient is clipped to bound the influence of any one sample, then Gaussian noise is added before the update is applied [5]. The result is an (ϵ, δ) guarantee rather than an informal promise, so privacy and utility can be discussed with a common vocabulary.

C. Model Inversion Attacks

Model inversion attacks try to reconstruct input attributes or approximate training examples from a trained model. Fredrikson et al. demonstrated that if an attacker can optimise against a model’s response, they may recover surprisingly informative approximations of sensitive data [3]. In a federated setting, the attack surface is smaller than in centralized learning, but it does not disappear, because the released model parameters can still reveal what the network has learned.

D. Face Recognition with Deep Embeddings

Face recognition systems rarely classify directly from pixels. They learn an embedding space in which the same person maps to nearby vectors and different people map far

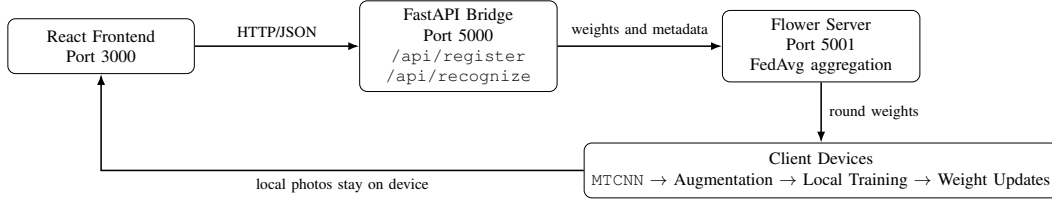


Fig. 1. The deployment is intentionally model-centric: the frontend talks to a FastAPI layer, FastAPI coordinates the federated service, and the clients keep their raw biometric data local while only model updates move upstream.

apart. FaceNet made this idea practical, and InceptionResNetV1 remains a strong embedding backbone because it combines Inception-style multi-scale filtering with residual connections and pretrained face features [4]. Recognition then becomes a distance or similarity comparison rather than a closed-set class prediction.

III. SYSTEM DESIGN AND ARCHITECTURE

A. Overview

The system follows a model-centric architecture. Raw photos stay on the client device, preprocessing turns them into aligned face tensors, local training updates a shared embedding model, and the server aggregates only weights. A FastAPI layer bridges the machine-learning pipeline with the React frontend so the user interface can trigger enrolment, recognition, federated rounds, and attack demonstrations. Figure 1 shows the deployment layout used in the project.

B. Two-Model Strategy

The project trains two checkpoints on the same code path. Version A is the vulnerable baseline without differential privacy, and Version B adds DP-SGD on top of the same federated procedure. This split is necessary because a privacy claim is only meaningful if the baseline is actually vulnerable and the protected model is actually harder to attack. Table I summarises the main configuration values taken from `config.py`.

C. Privacy Version Design

The frontend and API expose two privacy modes. In version 1, the server stores embeddings and performs recognition centrally against the stored templates. In version 2, the server does not keep embeddings and recognition must remain local on the device. The mode is controlled by the `PRIVACY_VERSION` variable in `linking/api.py` (can be changed via terminal using `"set PRIVACY_VERSION=2"`), which lets the same

TABLE I. MAIN CONFIGURATION VALUES USED TO DEFINE THE TWO TRAINING VERSIONS AND THE ATTACK EVALUATION.

Setting	Value
NUM_ROUNDS	20
MIN_CLIENTS	5
LOCAL_EPOCHS	5
LEARNING_RATE	10^{-4}
ATTACK_ITERATIONS	1000
ATTACK_LR	0.01
NOISE_MULTIPLIER	1.1
MAX_GRAD_NORM	1.0
DELTA	10^{-5}
EPSILON_MAX	5.0
DP_RANDOM_SEED	42
EMBEDDING_SIZE	512
THRESHOLD	0.6
SERVER_ADDRESS	127.0.0.1:8080

user interface demonstrate both the server-side and local-only privacy story without changing the underlying model.

IV. IMPLEMENTATION PIPELINE

A. Face Detection and Preprocessing

Every image entering the pipeline — whether during training, enrollment, or live recognition — first passes through MTCNN [8]. MTCNN operates as a three-stage cascade. The Proposal Network (P-Net) rapidly scans the image at multiple scales, generating a large set of candidate face regions. The Refine Network (R-Net) filters false positives from P-Net’s output. The Output Network (O-Net) produces the final bounding box coordinates together with five facial landmark positions: left eye, right eye, nose tip, and both mouth corners.

These landmarks are critical for *geometric alignment*: the image is rotated so that the two eyes lie on a horizontal axis, removing head-tilt variation that would otherwise cause the same person to produce dissimilar embeddings across photos. The aligned region is then cropped and resized to exactly 160×160 pixels with a 20-pixel margin. Pixel values are normalised to the range $[-1, 1]$ as required by the pretrained backbone:

$$x' = \frac{x - 127.5}{128}. \quad (1)$$

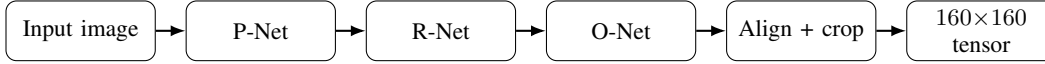


Fig. 2. MTCNN preprocessing cascade. Candidate face regions are proposed by P-Net, refined by R-Net, then precisely localised and landmark-aligned by O-Net before cropping and normalisation yield a 160×160 tensor ready for the embedding backbone.

Before MTCNN runs, each image undergoes a blur quality check using the Laplacian variance. Images with variance below 100 are rejected as too blurry for reliable face detection. All processed face tensors are saved as `.pt` files (PyTorch format), preserving exact 32-bit floating-point values without the lossy compression of JPEG. In practice, 2 out of 3 collected photographs per person were successfully processed; the third was typically rejected by the blur check and required retaking.

Figure 2 summarises the cascade.

B. Data Augmentation

The augmentation module exists because each identity starts with only a few photos. Without augmentation, this would lead to severe overfitting. The `FaceAugmentation` class applies a stochastic pipeline to each forward pass during training, generating diverse synthetic variants from each source image without saving them to disk. Specifically, each batch element undergoes horizontal flip (probability 0.5), small affine transformation (rotation $\pm 5^\circ$, translation fraction 0.02, scale range $[0.95, 1.05]$, shear 2°), and photometric jitter (brightness, contrast, saturation, and hue perturbations of 0.2, 0.2, 0.2, and 0.05 respectively). The result is that 3 source photographs effectively present as approximately 60 distinct training samples per person across the 5 local training epochs per FL round.

Critically, augmentation is applied *only during training*. At inference time — during recognition and enrollment — the raw normalised tensor is used directly, ensuring embedding stability.

C. Feature Extraction

We use InceptionResNetV1 [4], pretrained on the VGGFace2 dataset, as the embedding backbone. It processes each 160×160 face tensor and outputs a 512-dimensional vector that is L2-normalised to the unit hypersphere. This design choice reflects the model’s documented face verification accuracy of 99.6% on the LFW benchmark, substantially outperforming lighter alternatives such as MobileNetV2 (approximately 92%).

The embedding geometry is central to the recognition mechanism. Two photographs of the same person produce embeddings that are close in the 512-dimensional space

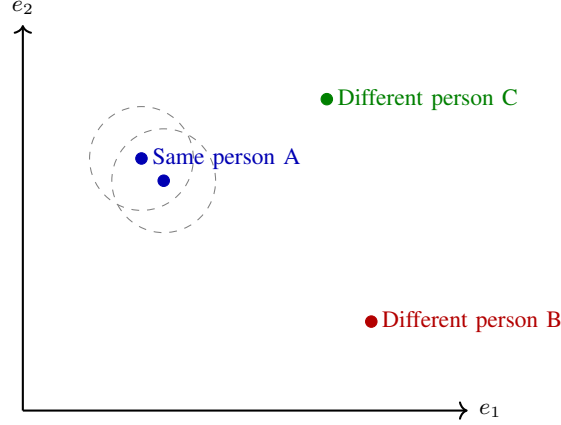


Fig. 3. Schematic of the 512-dimensional embedding space (projected to 2-D for illustration). Photos of the same identity cluster together (blue points, dashed circle), while photos of different identities (red, green) are pushed far apart. Recognition is implemented as a distance threshold on this geometry, not a classifier head.

(Euclidean distance below the recognition threshold of 0.6), while photographs of different people produce embeddings that are geometrically far apart. Recognition reduces entirely to this distance comparison: during enrollment, 1–3 photographs are processed into embeddings, averaged into a single template vector, and stored locally. During recognition, a live frame is embedded and compared to all stored templates; the closest match below the threshold is returned as the identity.

The model is used in embedding mode (`classify=False`), outputting raw descriptors rather than class scores. During FL training, it serves as the core of the local objective function rather than as a standalone classifier.

Figure 3 illustrates the intended geometry of the embedding space.

D. Federated Learning

Federated Learning orchestrates distributed training without data sharing. In each round, the Flower server sends the current global model weights to all five clients. Each client loads these weights, trains locally for `LOCAL_EPOCHS=5` epochs on its own augmented face data, and returns the updated weights together with its sample count. The server computes a weighted average — Federated Averaging [1]:

$$W_{\text{global}} = \sum_{k=1}^K \frac{n_k}{\sum_j n_j} W_k, \quad (2)$$

where W_k are the weights returned by client k and n_k is its local sample count. This process repeats for NUM_ROUNDS=20 rounds. At no point do face photographs, embeddings, or identity labels cross the network boundary — only the numerical parameter tensors travel.

The local training objective is a self-supervised embedding consistency loss. For each face tensor x and an augmented version $T(x)$, the client minimises:

$$\mathcal{L} = \frac{1}{|B|} \sum_{i \in B} \|f_\theta(x_i) - f_\theta(T(x_i))\|_2^2, \quad (3)$$

where f_θ is the embedding network and B is the local batch. This teaches the model that the same person’s face should produce consistent embeddings regardless of lighting, angle, or blur variation. The Adam optimiser with learning rate 10^{-4} is used throughout.

Since all five clients run simultaneously on a single student laptop, they are implemented as separate operating system processes via Python’s `multiprocessing` module, communicating with the Flower server over a local socket at `127.0.0.1:8080`.

E. The Frozen Backbone Limitation

The most important implementation limitation is that the InceptionResNetV1 backbone had to be frozen to make the workload fit student laptops. Running five simultaneous client processes with Opacus DP-SGD pushed memory usage beyond what was realistic for the hardware. DP-SGD is especially expensive because per-sample gradient accounting multiplies the memory footprint by the batch size, and the full backbone has about 23 million parameters. The practical fix was to keep the pretrained backbone fixed and train only the final projection layers. That reduced the trainable footprint enough for the experiment to run, but it also meant that federated training did not meaningfully improve the model across rounds.

Figure ?? confirms this: the flat local accuracy and loss curves show that the system is mostly preserving a strong pretrained embedding rather than learning a better one through FL.

V. PRIVACY AND SECURITY ARCHITECTURE

A. Differential Privacy

Differential Privacy [2], [5] provides a formal mathematical guarantee that an adversary observing the trained model cannot infer whether any specific individual’s data was included in training, beyond a bound parameterised by ϵ . Lower ϵ implies stronger privacy at the cost of model utility.

DP-SGD is implemented via Opacus, which intercepts the standard backward pass and applies two mechanisms on each training step. First, per-sample gradients are clipped to prevent any single photograph from dominating the weight update:

$$\tilde{g}_i = g_i \cdot \min\left(1, \frac{C}{\|g_i\|_2}\right), \quad (4)$$

where $C=1.0$ is the clipping norm. Second, calibrated Gaussian noise is added to the batch aggregate before the weight update:

$$\hat{g} = \frac{1}{B} \left(\sum_{i \in B} \tilde{g}_i + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I}) \right), \quad (5)$$

with noise multiplier $\sigma=1.1$. The privacy budget is tracked by Opacus’s accountant after each training step.

The system defines four preset privacy configurations ranging from *weak* ($\sigma=0.5$, target $\varepsilon \leq 10$) to *very strong* ($\sigma=1.5$, target $\varepsilon \leq 3$). Version B uses the *strong* preset ($\sigma=1.1$, $\varepsilon_{\max}=5.0$). In practice, the logged cumulative epsilon reached 10.21 across the evaluated runs, exceeding the configured cap. This indicates that the hard-stop enforcement was not correctly wired during these runs and is discussed further in Section IX.

B. Lightweight Manual DP Implementation and Memory Optimization

During implementation, a significant engineering challenge was encountered: the memory overhead of the standard Opacus DP-SGD module. Opacus tracks per-sample gradients for every trainable parameter, which, for the 23-million-parameter Inception-ResNetV1 backbone, exceeded the 8GB–16GB RAM capacity of student laptops when running multiple concurrent clients.

To resolve this, two strategies were implemented:

- 1) **Backbone Freezing:** The pretrained backbone was frozen, restricting training to the final 512-dimensional projection head (approximately 260,000 parameters). This reduced the memory footprint by roughly 100×, enabling stable multi-client execution.
- 2) **Manual DP-SGD Module:** A lightweight, “from-scratch” DP wrapper was developed as a fallback and technical proof-of-concept. This module implements manual gradient clipping using `torch.nn.utils.clip_grad_norm_` and Gaussian noise injection directly into the `optimizer.step()` loop, bypassing the heavy autograd hooks required by professional-grade libraries.

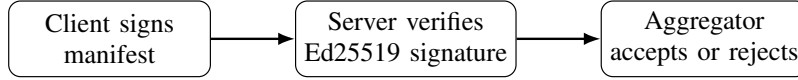


Fig. 4. Ed25519 signing workflow. The client signs a manifest that binds the round number, client ID, sample count, and a hash of the model parameters. The server verifies the signature before allowing any update into the aggregation step; invalid or missing signatures cause the update to be rejected and the event logged.

C. Ed25519 Digital Signatures

Every client signs its update manifest before sending the model update back to the server. The manifest contains the round index, client ID, sample count, parameter hash, and protocol version. The server checks the signature and rejects updates that are missing, malformed, or tied to the wrong protocol version. This protects integrity and authenticity: the server can tell whether the update really came from the claimed client and whether the payload was modified in transit.

Figure 4 illustrates the signing workflow.

D. AES-256-GCM Encryption

The repository also implements an encryption probe using ephemeral X25519 key agreement, HKDF-derived keys, and AES-256-GCM. The Additional Authenticated Data binds the round and client identity to the payload so replay or cross-round reuse is detectable. The important operational note is that this is currently validated as a probe rather than a full Flower transport replacement, so the primitives are present and checked, but the actual model update transport is still not encrypted end-to-end.

E. Tamper-Evident Audit Logging

Audit logging is hash-chained. Each log entry stores its own hash and the hash of the previous entry, so editing any earlier line breaks the chain from that point onward. The server and clients both persist audit logs for privacy-relevant events such as FL rounds, DP activation, epsilon updates, model checkpoints, and signature validation. This gives the project a forensic trail without pretending to provide network-layer secrecy.

F. Secure Aggregation Status

The code contains secure-aggregation scaffolding, but the current Flower version expects the newer ServerApp and ClientApp workflow. That means the legacy strategy path falls back gracefully and emits a warning instead of claiming a stronger guarantee

than the code can currently deliver. The right reading is therefore status, not completion: the migration path is identified, but it is not yet fully finished.

VI. ATTACK EVALUATION

A. Threat Model

The attacker in this evaluation is deliberately limited to the released global weights. The attacker does not receive the training photos, the per-client directories, the enrolment data, or the server-side embeddings. That assumption is realistic enough to matter: once a model is deployed, motivated adversaries can often obtain a copy of the checkpoint or query enough of the system to reconstruct it.

B. Model Inversion Methodology

The inversion attack starts from a random image and updates only the pixels. The model weights stay frozen, and the optimiser tries to move the image so that the model embedding becomes closer to the target identity’s embedding. This is standard input-space optimisation: the attack is not retraining the model, it is searching for an input that the model maps to the desired representation. In the project, Version A gives the optimiser clean gradients, so the reconstruction objective keeps dropping. Version B is trained with DP noise, so the learned landscape is noisier and the optimiser gets stuck at a much higher residual loss. The recovered image is therefore treated as evidence through the loss curve, not through photorealistic appearance.

C. Results and Analysis

Figure 5 is the central result of the paper. The blue baseline curve falls from 10^0 to about 10^{-4} over 200 iterations, while the red DP curve drops early and then plateaus around 10^{-2} . Because lower loss means the attacker is closer to the target embedding, the final gap is a privacy win for the protected model. The separation is roughly two orders of magnitude, which is large enough to be visible even on a logarithmic axis.

The bar chart next to it makes the same point in a form that is easier to explain to non-specialists. For Aaron_Peirsol, Version A ends at 2.2887×10^{-4} and Version B ends at 2.2556×10^{-2} . For Abdel_Nasser_Assidi, the corresponding values are 1.1744×10^{-4} and 1.9404×10^{-2} . The important detail is the consistency across identities: the protected model stays about 100 times harder to optimise against than the baseline.

Figure 6 adds two more pieces of evidence. The positive loss gap means the DP model remains harder to attack throughout the optimisation process, not just at the end. The 1000-iteration baseline run also shows that Version A was still improving long after the

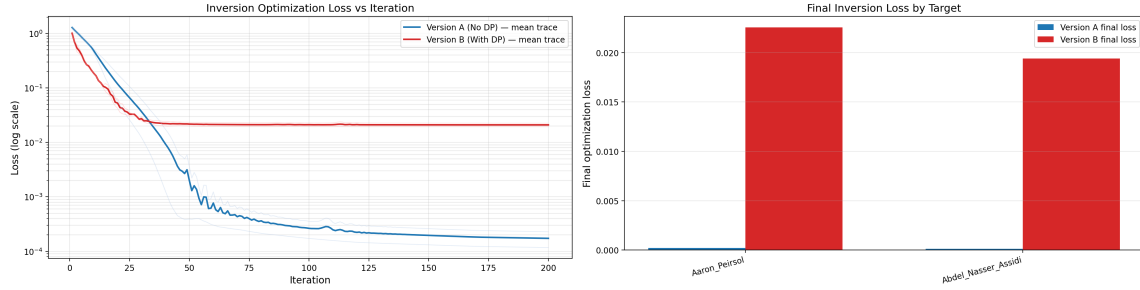


Fig. 5. Left: inversion optimisation loss on a log scale over 200 iterations. Blue (Version A, no DP) converges steadily to $\approx 10^{-4}$, meaning the attacker successfully approximated the target embedding. Red (Version B, with DP) drops initially then hard-plateaus at $\approx 10^{-2}$ because DP noise in the model weights prevents the optimiser from finding a consistent descent direction. The two-order-of-magnitude gap at convergence is the primary evidence of privacy protection. Right: bar chart of final inversion loss per identity. Version A bars (near zero) are barely visible; Version B bars are $\approx 100\times$ taller for both Aaron_Peirsol and Abdel_Nasser_Assidi, confirming the separation is not a one-off artefact.

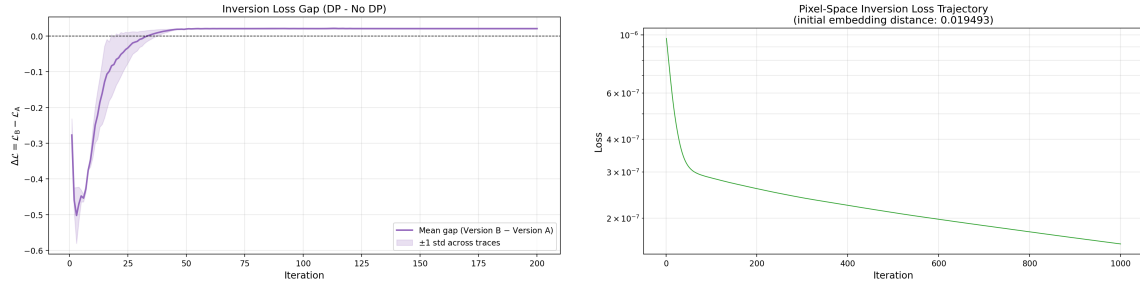


Fig. 6. Left: loss gap $\Delta\mathcal{L} = \mathcal{L}_B - \mathcal{L}_A$ across iterations, averaged over two target identities (shaded band = ± 1 std). A persistent positive gap after iteration 50 confirms that the DP model is consistently harder to invert throughout the optimisation, not only at convergence. Right: extended 1000-iteration inversion run on Version A only. The loss continues to fall monotonically to $\approx 1.6 \times 10^{-7}$, showing that the 200-iteration comparison window understates the full convergence depth of the unprotected model.

shorter comparison window ended, with the residual dropping to about 1.5950×10^{-7} . That extended run matters because it shows that the 200-iteration evaluation is conservative for the unprotected model.

Figure ?? addresses the privacy accountant output. All data points share the same logged epsilon value of 10.21; the scatter over pseudo-accuracy reflects within-round variation rather than a genuine privacy sweep.

VII. FRONTEND AND API INTEGRATION

The React frontend communicates with the Python back end through a FastAPI bridge. Three core endpoints handle the system’s main functions: `/api/register` receives a set of face images, processes them through MTCNN and the global model, averages the resulting embeddings, and stores the identity; `/api/recognize` compares a submitted embedding against all stored templates using cosine similarity with threshold 0.6; and `/api/fl-results` returns training metadata for display on the dashboard screen.

The system supports two privacy modes, configurable via an environment variable. In Privacy Version 1, embeddings are stored on the central server and recognition is performed server-side. In Privacy Version 2, embeddings are stored locally on the client device and recognition is performed locally using the globally distributed model, with no embedding data sent to the server. This mode represents the stronger privacy guarantee and is the intended production configuration.

The frontend was actively tested for face recognition, with enrollment and live recognition confirmed to operate correctly using the pretrained backbone.

VIII. APPLICATION OVERVIEW

The system is built as a full-stack face recognition application combining a React-based frontend with a FastAPI backend and a federated learning server. The user interacts with the system through a simple interface that supports face enrollment, real-time recognition.

A. Application Workflow Illustration

The following figures illustrate the main screens of the application and how a user interacts with the system during enrollment, recognition, and monitoring phases.

Register Recognize

Face Registration

Your photos stay on this device. Only embeddings participate in FL.

Full Name

Step 1/3 0/3 captured

Look straight

Upload Camera

Back Next

Register Face

Fig. 7. Register Section. The user scans their face from three different angles (front, left, and right). Each image is processed locally, aligned using MTCNN, and converted into embeddings for secure storage.

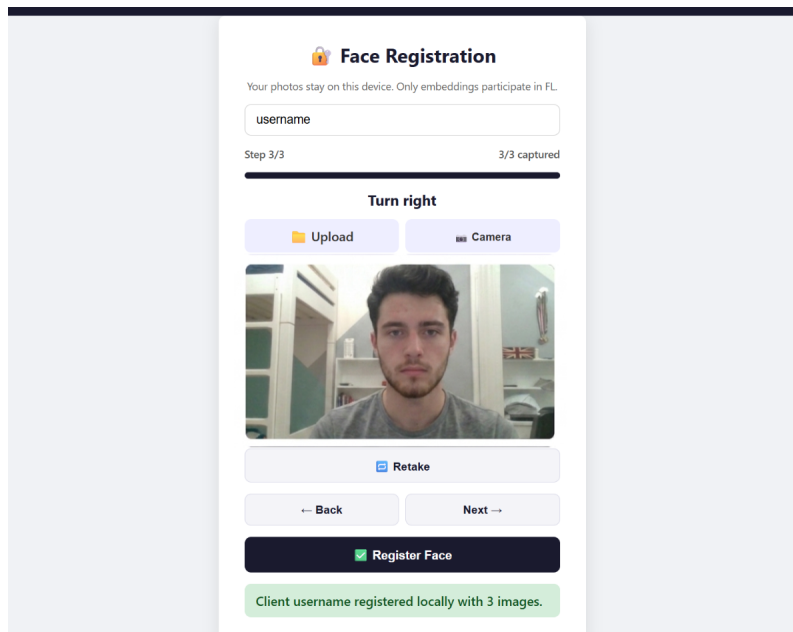


Fig. 8. Post-Registration Recognition Screen. After completing face registration, the user becomes part of the global model and can be recognized during inference. The system matches live input against federated embeddings trained across all clients.

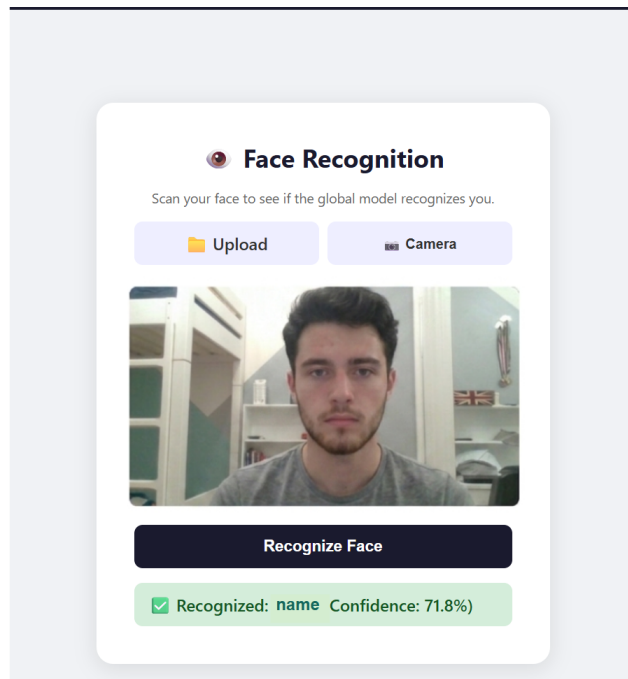


Fig. 9. Recognition Section. The user uploads a face image for identification. The model compares the input against stored federated embeddings and successfully recognizes the user with a confidence score of 71.8%. The reduction from an ideal 100% confidence is attributed to data augmentation and the addition of differential privacy noise, which introduce controlled perturbations to improve generalization and privacy protection.

IX. ENCOUNTERED ISSUES AND LIMITATIONS

Backbone freeze and FL convergence. The most significant limitation of this implementation is the backbone freeze described in Section IV-E. Running five Opacus-wrapped Flower clients simultaneously on student hardware exceeded available RAM. The consequence is that FL training does not improve the model: the federated aggregation loop is structurally operational but functionally inert. Resolving this requires either GPU resources for gradient checkpointing, or a staged training protocol that runs clients sequentially rather than concurrently.

Epsilon cap not enforced. The privacy accountant configuration specifies $\epsilon_{\max}=5.0$ as a hard training cutoff. The logged cumulative epsilon reached 10.21, double the intended cap. The accountant query was operational, but the training loop was not wired to halt when the budget was exceeded. The formal privacy guarantee of this run is therefore $\epsilon=10.21$, not $\epsilon=5.0$.

AES encryption not connected to transport. The AES-256-GCM encryption primitives are implemented and tested in isolation. Flower parameter payloads currently travel in plaintext because wiring encryption into the serialisation layer requires a more invasive integration not yet completed.

Secure Aggregation pending migration. The SecAgg+ strategy requires Flower’s newer ServerApp/ClientApp API. The current codebase uses the legacy strategy interface, and migration is a planned but incomplete step.

Per-round global accuracy not logged. The Flower `History` object containing per-round server-aggregated accuracy was not serialised to disk. A global FL accuracy curve cannot be reconstructed from existing data.

X. FUTURE WORK

The first next step is to remove the backbone freeze once better hardware or gradient checkpointing is available. That would let the federated training loop actually refine the embedding model instead of only preserving a pretrained baseline. It would also make the local accuracy and loss curves a real learning signal instead of a limitation signal.

The second step is to finish the Flower migration needed for secure aggregation. That work would make the current warning-based fallback disappear and would give the project a stronger confidentiality story on the transport side.

The third step is to wire the AES-256-GCM layer into the actual model-update path rather than only probing it. At the moment the primitive is correct, but the transport integration remains a separate engineering task.

The fourth step is to enforce the privacy budget correctly. The accountant already measures epsilon, so the remaining problem is to stop training when the configured threshold is crossed and to record that stop clearly in the logs.

The final step is to extend the attack study. More identities, more iterations, and a better generative prior would make the privacy evaluation more complete, especially once the runtime constraints are relaxed.

XI. CONCLUSION

This project delivers a working privacy-preserving face recognition pipeline built around federated learning, differential privacy, digital signatures, audit logging, and attack evaluation. The strongest quantitative result is the inversion-loss gap: the DP model stays about 100 times harder to optimise against than the baseline on two different target identities, and the baseline continues to improve even when the optimisation budget is extended to 1000 iterations.

The report is also explicit about what the implementation does not yet achieve. The frozen backbone keeps the system within the memory budget of student laptops, but it prevents meaningful federated improvement, and the privacy accountant exceeded the configured epsilon cap. Those are real limitations, but they do not cancel the privacy demonstration. They define the boundary between what was proven, what was approximated, and what remains for future work.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. AISTATS*, 2017.
- [2] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating Noise to Sensitivity in Private Data Analysis," in *Proc. TCC*, 2006.
- [3] M. Fredrikson, S. Jha, and T. Ristenpart, "Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures," in *Proc. CCS*, 2015.
- [4] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *Proc. CVPR*, 2015.
- [5] M. Abadi *et al.*, "Deep Learning with Differential Privacy," in *Proc. CCS*, 2016.
- [6] P. Kairouz *et al.*, "Advances and Open Problems in Federated Learning," 2021.
- [7] N. Rodriguez-Barroso *et al.*, "A Tutorial on Federated Learning from Theory to Practice," *IEEE/CAA Journal of Automatica Sinica*, vol. 11, no. 4, 2024.
- [8] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016.